

Методические рекомендации по элементам WPF

Оглавление

Кнопки	2
Элементы отображения данных в WPF	5
Работа с элементами выбора дат	9
Работа с элементами выбора	13

Кнопки

Элемент Button

Button представляет собой стандартную командную кнопку, которая реагирует на щелчок мышью или нажатие клавиш (Enter, Пробел). При нажатии и отпускании кнопки (или при однократном щелчке, в зависимости от режима) происходит событие **Click**, которое используется для выполнения логики приложения.

Основные свойства и события

Команда	Тип	Описание
Content	свойство	Определяет отображаемый текст или визуальный элемент внутри кнопки.
Command	свойство	Позволяет привязать команду (реализующую интерфейс ICommand), которая будет выполняться при нажатии.
CommandParameter	свойство	Параметр, передаваемый в команду.
IsEnabled	свойство	Указывает, доступна ли кнопка для взаимодействия.
ClickMode	свойство	Определяет, когда возникает событие Click: при нажатии (Press), отпускании (Release) или наведении (Hover).
Click	событие	Возникает при активации кнопки в соответствии с ClickMode.

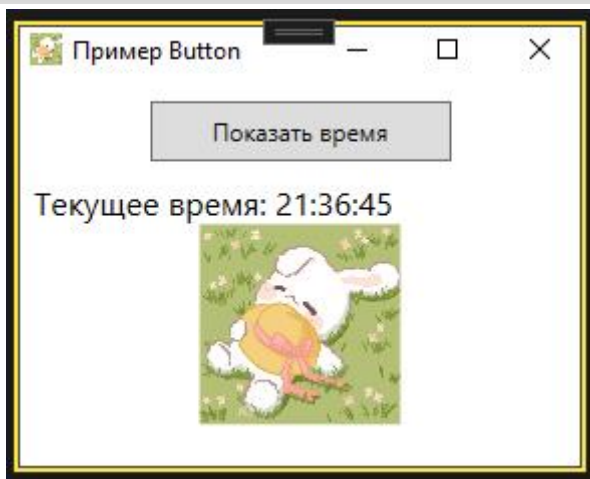
Задача: создать кнопку, которая при нажатии выводит в текстовый блок текущее время.

XAML-разметка:

```
<Window x:Class="WpfApp6.MainWindow"
  xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
  xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
  xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
  xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
  xmlns:local="clr-namespace:WpfApp6"
  mc:Ignorable="d"
  Icon="/123.png"
  Title="Пример Button" Height="200" Width="300">
  <StackPanel Margin="10">
    <Button x:Name="ShowTimeButton"
      Content="Показать время"
      Width="150" Height="30"
      Click="ShowTimeButton_Click"/>
    <TextBlock x:Name="TimeTextBlock" Margin="0,10,0,0" FontSize="16"/>
    <Image Source="/123.png" Width="100" Height="100"/>
  </StackPanel>
</Window>
```

Обработчик события в C#:

```
private void ShowTimeButton_Click(object sender, RoutedEventArgs e)
{
    TimeTextBlock.Text = "Текущее время: " + DateTime.Now.ToLongTimeString();
}
```



При установке **ClickMode="Hover"** событие будет срабатывать при наведении курсора, что может быть полезно для создания интерактивных подсказок.

Свойство **IsEnabled** позволяет динамически блокировать кнопку, например, пока не заполнены все поля формы.

Элемент RepeatButton

RepeatButton – это специализированная кнопка, которая непрерывно генерирует событие **Click** через заданные промежутки времени, пока кнопка удерживается нажатой. Это удобно для элементов управления, требующих многократного повторения действия (например, регуляторы громкости, счётчики, кнопки прокрутки).

Основные свойства и события

Команда	Тип	Описание
Delay	свойство	Время в миллисекундах от момента нажатия до первого повторного события.
Interval	свойство	Период в миллисекундах между последующими событиями при удержании.
Click	событие	Возникает однократно при первом нажатии, а затем многократно с интервалом, заданным Interval.

Задача: создать счётчик, значение которого увеличивается при удержании кнопки «+» и уменьшается при удержании кнопки «-».

XAML-разметка:

```
<Window x:Class="WpfApp6.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
        xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
        xmlns:local="clr-namespace:WpfApp6"
        mc:Ignorable="d"
        Icon="/123.png"
        Title="Пример" Height="200" Width="300">
    <StackPanel Orientation="Horizontal" HorizontalAlignment="Center"
        VerticalAlignment="Center">
        <RepeatButton x:Name="DecrementButton"
            Width="40" Height="40" Content="-"
            Delay="500" Interval="100"
            Click="DecrementButton_Click"/>
        <TextBlock x:Name="CounterTextBlock"
            Text="0" FontSize="24" Width="50"
            TextAlignment="Center" VerticalAlignment="Center"/>
        <RepeatButton x:Name="IncrementButton"
            Width="40" Height="40" Content="+"
            Delay="500" Interval="100"
            Click="IncrementButton_Click"/>
    </StackPanel>
</Window>
```

Обработчик события в C#:

```
private int _counter = 0;

private void IncrementButton_Click(object sender, RoutedEventArgs e)
{
    _counter++;
    CounterTextBlock.Text = _counter.ToString();
}

private void DecrementButton_Click(object sender, RoutedEventArgs e)
{
    _counter--;
    CounterTextBlock.Text = _counter.ToString();
}
```



RepeatButton наследуется от **ButtonBase**, поэтому поддерживает все свойства базового класса (например, **Content**, **Command**, **ClickMode**).

Свойства **Delay** и **Interval** особенно важны для обеспечения комфортного взаимодействия: слишком короткая задержка может привести к случайным срабатываниям, слишком длинная — к ощущению вялости интерфейса.

Рекомендуется использовать **RepeatButton** в сценариях, где требуется плавное изменение значения (например, ползунки времени, регуляторы яркости).

Элементы отображения данных в WPF

DataGrid идеально подходит для отображения табличных данных с возможностью редактирования, сортировки и группировки.

ListView используется для представления списков в различных видах (например, в виде таблицы с колонками или простого списка с иконками).

TreeView предназначен для отображения иерархических данных (деревьев), таких как файловая система, организационная структура и т.п.

Элемент DataGrid

DataGrid представляет собой элемент управления, который отображает данные в табличном формате с строками и столбцами. Он поддерживает редактирование, сортировку, фильтрацию, группировку и встроенную навигацию. **DataGrid** автоматически генерирует столбцы на основе структуры источника данных, но также позволяет полностью контролировать их определение вручную.

Основные свойства, методы и события

Методы	Тип	Описание
ItemsSource	свойство	Привязывает коллекцию данных для отображения.
AutoGenerateColumns	свойство	Определяет, следует ли автоматически создавать столбцы на основе свойств элементов коллекции (по умолчанию true).
CanUserAddRows	свойство	Разрешает пользователю добавлять новые строки (по умолчанию true).
CanUserDeleteRows	свойство	Разрешает пользователю удалять строки (по умолчанию true).
CanUserSortColumns	свойство	Разрешает сортировку по столбцам (по умолчанию true).
SelectedItem	свойство	Получает или задаёт выбранный элемент (объект строки).
SelectedItems	свойство	Возвращает коллекцию выбранных элементов (при множественном выборе).
SelectionMode	свойство	Режим выбора: Single, Extended (множественный с модификаторами).
ColumnDefinitions	коллекция	Позволяет вручную определить столбцы (например, DataGridTextColumn, DataGridCheckBoxColumn).
SelectionChanged	событие	Возникает при изменении выделения.
CellEditEnding	событие	Возникает при завершении редактирования ячейки (можно отменить или подтвердить изменения).
LoadingRow	событие	Позволяет настроить внешний вид строки при её создании (например, установить номер строки).

Задача: отобразить список сотрудников (имя, возраст, должность) с возможностью редактирования и сортировки.

Обработчик события в C#:

```
public class Employee
{
    public string Name { get; set; }
    public int Age { get; set; }
    public string Position { get; set; }
}
```

XAML-разметка:

```
<Grid>
    <DataGrid x:Name="EmployeesDataGrid"
        AutoGenerateColumns="True"
        CanUserAddRows="True"
        CanUserDeleteRows="True"
        CanUserSortColumns="True"
        SelectionMode="Single"
        Margin="10"/>
</Grid>
```

Обработчик события в C#:

```
public partial class MainWindow : Window
{
    public MainWindow()
    {
        InitializeComponent();

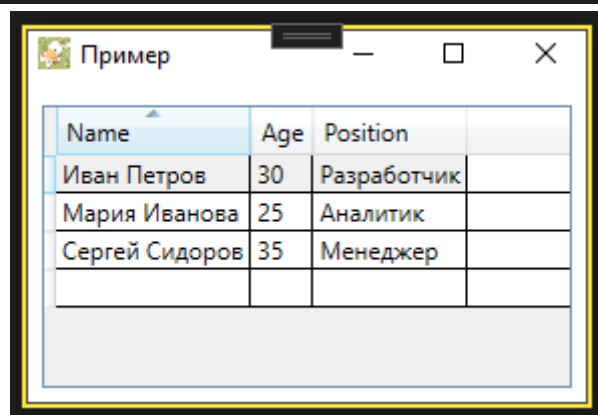
        var employees = new List<Employee>
        {
            new Employee { Name = "Иван Петров", Age = 30, Position = "Разработчик" },
            new Employee { Name = "Мария Иванова", Age = 25, Position = "Аналитик" },
            new Employee { Name = "Сергей Сидоров", Age = 35, Position = "Менеджер" }
        };

        EmployeesDataGrid.ItemsSource = employees;
    }
}
```

```

16 namespace WpfApp6
17 {
18     /// <summary>
19     /// Логика взаимодействия для MainWindow.xaml
20     /// </summary>
21     public partial class MainWindow : Window
22     {
23         public class Employee
24         {
25             public string Name { get; set; }
26             public int Age { get; set; }
27             public string Position { get; set; }
28         }
29
30         public MainWindow()
31         {
32             InitializeComponent();
33             var employees = new List<Employee>
34             {
35                 new Employee { Name = "Иван Петров", Age = 30, Position = "Разработчик" },
36                 new Employee { Name = "Мария Иванова", Age = 25, Position = "Аналитик" },
37                 new Employee { Name = "Сергей Сидоров", Age = 35, Position = "Менеджер" }
38             };
39             EmployeesDataGrid.ItemsSource = employees;
40         }
41     }
42 }

```



Свойство **ItemsSource** привязано к коллекции **employees**. DataGrid автоматически создаст столбцы для свойств Name, Age, Position.

Благодаря **CanUserAddRows** и **CanUserDeleteRows** пользователь может добавлять новые строки (внизу появится пустая строка) и удалять выделенные строки клавишей Delete.

Сортировка доступна по щелчку на заголовке столбца.

Также можно вручную определить столбцы через Binding:

XAML-разметка:

```

<DataGrid x:Name="EmployeesDataGrid" AutoGenerateColumns="False">
    <DataGrid.Columns>
        <DataGridTextColumn Header="Имя" Binding="{Binding Name}" Width="*"/>
        <DataGridTextColumn Header="Возраст" Binding="{Binding Age}" Width="80"/>
        <DataGridTextColumn Header="Должность" Binding="{Binding Position}" Width="*"/>
    </DataGrid.Columns>
</DataGrid>

```

Для отмены автоматической генерации столбцов используется **AutoGenerateColumns="False"**.

Для реактивного обновления данных при изменении коллекции рекомендуется использовать `ObservableCollection<T>` вместо `List<T>`.

Для валидации данных можно реализовать интерфейс `IDataErrorInfo` в модели или обрабатывать событие `CellEditEnding`.

Элемент **ListView**

ListView — это элемент управления, который отображает список элементов данных. Он является наследником **ListBox** и добавляет поддержку настраиваемого представления через свойство **View**. Наиболее часто используемое представление — **GridView**, которое позволяет отображать данные в виде таблицы с колонками. Однако **ListView** также может использоваться с произвольными шаблонами элементов (**ItemTemplate**) для создания списков с иконками, изображениями и т.д.

Основные свойства, методы и события

Методы	Тип	Описание
ItemsSource	свойство	Привязывает коллекцию данных для отображения.
View	свойство	Задаёт объект представления (например, GridView), определяющий внешний вид элементов.
SelectedItem	свойство	Получает или задаёт выбранный элемент.
SelectionMode	свойство	Режим выбора (Single, Multiple, Extended).
ItemTemplate	свойство	Шаблон данных для отображения каждого элемента (если не используется View).
SelectionChanged	событие	Возникает при изменении выделения.

Задача: отобразить список задач (название, статус, приоритет) в виде таблицы с колонками.

XAML-разметка:

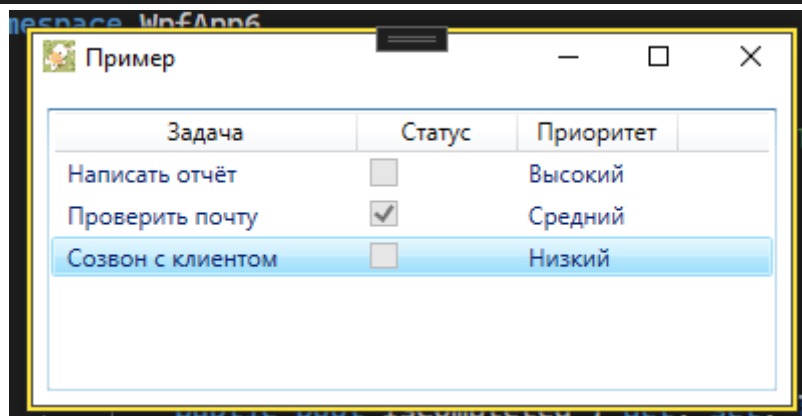
```
<Grid>
    <ListView x:Name="TasksListView" Margin="10">
        <ListView.View>
            <GridView>
                <GridViewColumn Header="Задача" DisplayMemberBinding="{Binding Title}" Width="150"/>
                <GridViewColumn Header="Статус" Width="80">
                    <GridViewColumn.CellTemplate>
                        <DataTemplate>
                            <CheckBox IsChecked="{Binding IsCompleted}"
                                IsEnabled="False"/>
                        </DataTemplate>
                    </GridViewColumn.CellTemplate>
                </GridViewColumn>
                <GridViewColumn Header="Приоритет" DisplayMemberBinding="{Binding Priority}" Width="80"/>
            </GridView>
        </ListView.View>
    </ListView>
</Grid>
```



```

16 namespace WpfApp6
17 {
18     /// <summary>
19     /// Логика взаимодействия для MainWindow.xaml
20     /// </summary>
21     public partial class MainWindow : Window
22     {
23         public class TaskItem
24         {
25             public string Title { get; set; }
26             public bool IsCompleted { get; set; }
27             public string Priority { get; set; }
28         }
29
30         public MainWindow()
31         {
32             InitializeComponent();
33
34             var tasks = new List<TaskItem>
35             {
36                 new TaskItem { Title = "Написать отчёт", IsCompleted = false, Priority = "Высокий" },
37                 new TaskItem { Title = "Проверить почту", IsCompleted = true, Priority = "Средний" },
38                 new TaskItem { Title = "Созвон с клиентом", IsCompleted = false, Priority = "Низкий" }
39             };
40
41             TasksListView.ItemsSource = tasks;
42         }
43     }
44 }
45

```



ListView.View устанавливается в GridView, который содержит три колонки.

Для колонки «Статус» используется CellTemplate, чтобы отобразить CheckBox вместо текста. Свойство DisplayMemberBinding связывает отображаемый текст с нужным свойством модели. Коллекция привязывается через ItemsSource.

ListView наследуется от **ListBox**, поэтому поддерживает множественный выбор, прокрутку, виртуализацию.

В отличие от **DataGrid**, **ListView** не предоставляет встроенных возможностей редактирования (если требуется, нужно реализовывать вручную через шаблоны).

Для простого списка с иконками можно использовать **ItemTemplate** без **GridView**.

Работа с элементами выбора дат

Элемент Calendar

Calendar представляет собой элемент управления, который отображает календарь на один месяц с возможностью переключения между месяцами и годами. Поддерживает выбор дат в различных режимах (**SingleDate**, **SingleRange**, **MultipleRange**). Позволяет подсвечивать сегодняшнюю дату, блокировать недоступные для выбора дни и настраивать первый день недели.

Основные свойства и события

Команда	Тип	Описание
SelectedDate	свойство	Получает или задаёт выбранную дату (тип DateTime?). Если выбран диапазон, содержит начальную дату.
SelectedDates	свойство	Коллекция выбранных дат (тип SelectedDatesCollection). Используется при SelectionMode = MultipleRange или SingleRange.
DisplayDate	свойство	Определяет отображаемый в календаре месяц/год. Не влияет на выбранную дату.
DisplayMode	свойство	Режим отображения: Month (месяц), Year (год), Decade (десятилетие).
SelectionMode	свойство	Режим выбора: SingleDate (одна дата), SingleRange (один диапазон), MultipleRange (произвольные даты), None (запрет выбора).
FirstDayOfWeek	свойство	День недели, с которого начинается отображение календаря (по умолчанию зависит от региональных настроек).
IsTodayHighlighted	свойство	Определяет, выделяется ли текущая дата (по умолчанию true).
BlackoutDates	коллекция	Позволяет задать диапазоны дат, которые будут недоступны для выбора (например, выходные или праздничные дни).
SelectedDatesChanged	событие	Возникает при изменении выбранной даты (дат).
DisplayDateChanged	событие	Возникает при изменении отображаемого месяца/года.
DisplayModeChanged	событие	Возникает при смене режима отображения (месяц/год/десятилетие).

Задача: создать календарь для выбора диапазона дат (например, периода отпуска) и отображать выбранный диапазон в текстовом блоке. Заблокировать для выбора все даты до сегодняшнего дня.

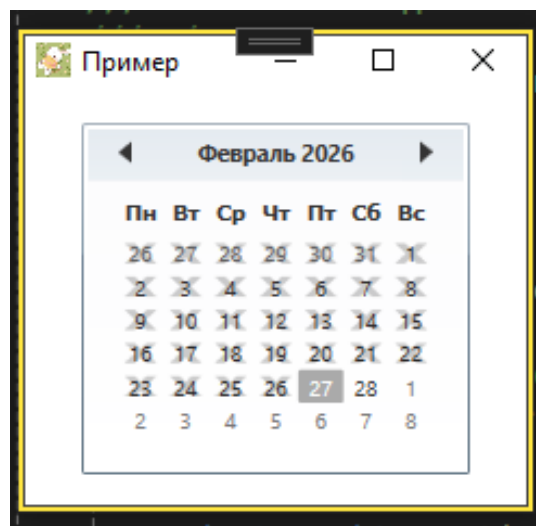
XAML-разметка:

```
<StackPanel Margin="10">
    <Calendar x:Name="VacationCalendar"
        SelectionMode="SingleRange"
        FirstDayOfWeek="Monday"
        IsTodayHighlighted="True"
        SelectedDatesChanged="VacationCalendar_SelectedDatesChanged"/>
    <TextBlock x:Name="RangeTextBlock" Margin="0,10,0,0" FontSize="14"/>
</StackPanel>
</Window>
```

```

21 public partial class MainWindow : Window
22 {
23
24     public MainWindow()
25     {
26         InitializeComponent();
27
28         // Запрещаем выбор дат до сегодняшнего дня
29         VacationCalendar.BlackoutDates.Add(new CalendarDateRange(DateTime.MinValue, DateTime.Today.AddDays(-1)));
30     }
31
32     private void VacationCalendar_SelectedDatesChanged(object sender, SelectionChangedEventArgs e)
33     {
34         if (VacationCalendar.SelectedDates.Count == 0)
35         {
36             RangeTextBlock.Text = "Диапазон не выбран";
37             return;
38         }
39
40         DateTime start = VacationCalendar.SelectedDates.First();
41         DateTime end = VacationCalendar.SelectedDates.Last();
42
43         if (start == end)
44             RangeTextBlock.Text = $"Выбрана одна дата: {start.ToShortDateString()}";
45         else
46             RangeTextBlock.Text = $"Выбран диапазон: с {start.ToShortDateString()} по {end.ToShortDateString()}";
47     }
48 }
49

```



SelectionMode="SingleRange" позволяет выбрать непрерывный диапазон дат.

BlackoutDates добавляет диапазон от минимальной даты до вчерашнего дня, делая эти даты недоступными для выбора (они отображаются серым).

В обработчике **SelectedDatesChanged** мы получаем первую и последнюю даты из коллекции **SelectedDates** и формируем строку для вывода. Коллекция содержит все даты выбранного диапазона.

При **SelectionMode = MultipleRange** пользователь может выбирать произвольные несмежные даты (например, отмечать праздники).

Для очистки выбора можно установить **SelectedDate = null** или вызвать **SelectedDates.Clear()**.

Свойство **DisplayDate** удобно использовать для программного перелистывания календаря к определённому месяцу.

Элемент DatePicker

DatePicker — это элемент управления, объединяющий текстовое поле для ввода даты и раскрывающийся календарь. Пользователь может ввести дату вручную или выбрать её из выпадающего календаря. Элемент автоматически проверяет корректность введённой даты и предоставляет свойства для форматирования отображения.

Основные свойства и события

Команда	Тип	Описание
SelectedDate	свойство	Получает или задаёт выбранную дату (тип DateTime?).
DisplayDate	свойство	Дата, которая отображается в раскрывающемся календаре (если календарь открыт). По умолчанию равна SelectedDate или сегодняшней дате.
DisplayDateStart / DisplayDateEnd	свойства	Ограничивают диапазон дат, доступных для выбора в календаре.
FirstDayOfWeek	свойство	Задаёт первый день недели в календаре.
IsDropDownOpen	свойство	Управляет состоянием раскрывающегося календаря (true — открыт, false — закрыт).
Text	свойство	Текстовое представление введённой даты (можно получить введённое пользователем значение, даже если оно некорректно).
SelectedDateFormat	свойство	Формат отображения даты в текстовом поле: Short или Long.
SelectedDateChanged	событие	Возникает при изменении выбранной даты (через календарь или ручной ввод).
CalendarOpened / CalendarClosed	события	Возникают при открытии и закрытии выпадающего календаря.

Задача: создать DatePicker для выбора даты рождения, с ограничением возраста от 18 до 100 лет (т.е. даты ранее 01.01.1923 и позже вчерашнего дня недоступны). При изменении даты выводить возраст в текстовый блок.

XAML-разметка:

```
<StackPanel Margin="10">
    <DatePicker x:Name="BirthDatePicker"
        SelectedDateFormat="Short"
        FirstDayOfWeek="Monday"
        SelectedDateChanged="BirthDatePicker_SelectedDateChanged"
        Margin="5"/>
    <TextBlock x:Name="AgeTextBlock" Margin="5" FontSize="14"/>
</StackPanel>
</Window>
```

Обработчик события в C#:

```
public MainWindow()
{
    InitializeComponent();

    // Ограничиваем выбор дат: от 100 лет назад до вчерашнего дня
    BirthDatePicker.DisplayDateStart = DateTime.Today.AddYears(-100);
    BirthDatePicker.DisplayDateEnd = DateTime.Today.AddDays(-1);
}

private void BirthDatePicker_SelectedDateChanged(object sender, SelectionChangedEventArgs e)
{
    if (BirthDatePicker.SelectedDate.HasValue)
    {
        DateTime selected = BirthDatePicker.SelectedDate.Value;
        int age = DateTime.Today.Year - selected.Year;
        // Корректировка, если день рождения ещё не наступил в этом году
        if (selected.Date > DateTime.Today.AddYears(-age)) age--;

        AgeTextBlock.Text = $"Возраст: {age} лет";
    }
    else
    {
        AgeTextBlock.Text = "Дата не выбрана";
    }
}
```

DisplayDateStart и **DisplayDateEnd** ограничивают доступные даты в выпадающем календаре, но пользователь всё равно может ввести дату вручную. Для полной блокировки нужно также проверять введённое значение или использовать валидацию.

Событие **SelectedDateChanged** срабатывает как при выборе из календаря, так и при ручном вводе (если введена корректная дата).

Возраст рассчитывается с учётом дня рождения.

Если пользователь вводит некорректную дату (например, "32.13.2023"), свойство **SelectedDate** остаётся прежним, а в **Text** можно получить введённую строку для анализа.

Для форматирования отображения даты в текстовом поле используется **SelectedDateFormat**, а также можно задать свойство **Language** для учёта региональных стандартов.

Для пустого значения (если дата не обязательна) установите **SelectedDate = null**.

Работа с элементами выбора

Элемент **CheckBox**

CheckBox представляет собой флажок, который может находиться в одном из трёх состояний: включён (отмечен), выключен (не отмечен) или неопределён (промежуточное состояние, обычно отображается затенённым). Неопределённое состояние используется, когда выбор не сделан или применим частично (например, в дереве, где отмечены не все дочерние элементы).

Основные свойства и события

Команда	Тип	Описание
IsChecked	свойство	Состояние флажка: true (включён), false (выключен), null (неопределён).

IsThreeState	свойство	Определяет, поддерживает ли флажок три состояния (true) или только два (false). По умолчанию false.
Content	свойство	Текст или визуальный элемент, отображаемый рядом с флажком.
Checked	событие	Возникает при переходе в состояние true.
Unchecked	событие	Возникает при переходе в состояние false.
Indeterminate	событие	Возникает при переходе в состояние null.

Задача: создать флажок для согласия с условиями и три состояния для опции "Уведомления" (Да/Нет/Не определено).

ХАМЛ-разметка:

```
<StackPanel Margin="10">
    <CheckBox x:Name="AgreeCheckBox"
        Content="Я согласен с условиями"
        Checked="AgreeCheckBox_Checked"
        Unchecked="AgreeCheckBox_Unchecked"/>
    <TextBlock x:Name="AgreeTextBlock" Margin="0,5,0,10"/>

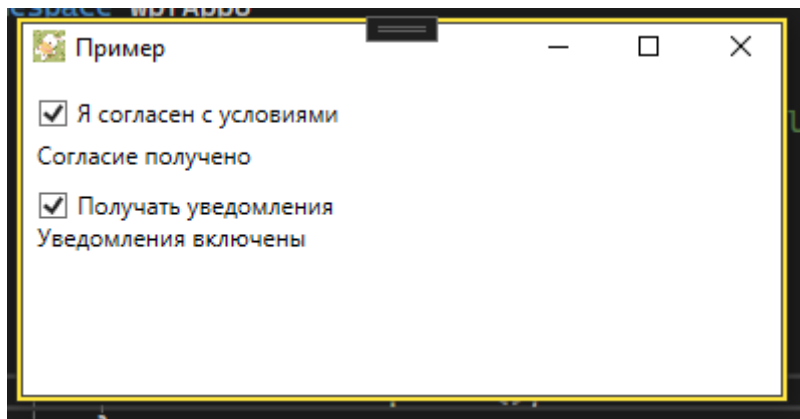
    <CheckBox x:Name="NotificationsCheckBox"
        Content="Получать уведомления"
        IsThreeState="True"
        Click="NotificationsCheckBox_Click"/>
    <TextBlock x:Name="NotificationsTextBlock"/>
</StackPanel>
</Window>
```

Обработчик события в C#:

```
private void AgreeCheckBox_Checked(object sender, RoutedEventArgs e)
{
    AgreeTextBlock.Text = "Согласие получено";
}

private void AgreeCheckBox_Unchecked(object sender, RoutedEventArgs e)
{
    AgreeTextBlock.Text = "Согласие не получено";
}

private void NotificationsCheckBox_Click(object sender, RoutedEventArgs e)
{
    switch (NotificationsCheckBox.IsChecked)
    {
        case true:
            NotificationsTextBlock.Text = "Уведомления включены";
            break;
        case false:
            NotificationsTextBlock.Text = "Уведомления отключены";
            break;
        case null:
            NotificationsTextBlock.Text = "Не определено (зависит от системных настроек)";
            break;
    }
}
```



Для первого флажка **IsThreeState** не задан (по умолчанию **false**), поэтому он работает только с **true/false**. События **Checked** и **Unchecked** обрабатываются отдельно.

Для второго флажка установлено **IsThreeState="True"**, поэтому возможны три состояния. Используем событие **Click**, чтобы реагировать на любое изменение.

Свойство **IsChecked** типа **bool?** удобно проверять через switch для трёх состояний.

Если **IsThreeState="True"**, щелчки по флажку циклически переключают состояния: **true** → **false** → **null** → **true** ... (порядок можно изменить, но стандартно так).

Элемент **RadioButton**

RadioButton — это переключатель, который обычно используется в группе для выбора одного варианта. Все **RadioButton** с одинаковым родительским контейнером и одинаковым значением свойства **GroupName** считаются одной группой, и только один из них может быть выбран одновременно. По умолчанию группа определяется общим родителем (например, **StackPanel**).

Основные свойства и события

Команда	Тип	Описание
IsChecked	свойство	Определяет, выбран ли переключатель (true / false).
GroupName	свойство	Задаёт имя группы для объединения переключателей, расположенных в разных контейнерах.
Content	свойство	Текст или элемент рядом с переключателем.
Checked	событие	Возникает, когда переключатель становится выбранным.
Unchecked	событие	Возникает, когда переключатель теряет выбор.

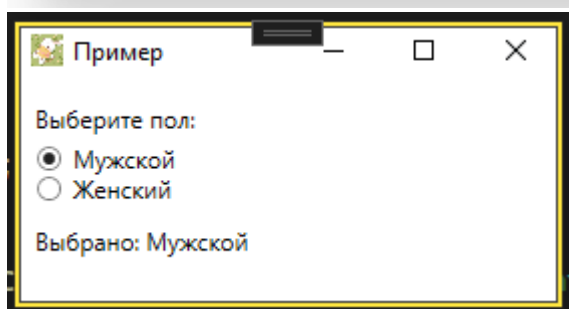
Задача: Создать группу выбора пола (Мужской/Женский) и отображать выбранное значение.

XAML-разметка:

```
<StackPanel Margin="10">
    <TextBlock Text="Выберите пол:" Margin="0,0,0,5"/>
    <RadioButton x:Name="MaleRadio" Content="Мужской" GroupName="Gender"
Checked="GenderRadio_Checked"/>
    <RadioButton x:Name="FemaleRadio" Content="Женский" GroupName="Gender"
Checked="GenderRadio_Checked"/>
    <TextBlock x:Name="GenderTextBlock" Margin="0,10,0,0"/>
</StackPanel>
</Window>
```


Обработчик события в C#:

```
private void GenderRadio_Checked(object sender, RoutedEventArgs e)
{
    RadioButton radio = sender as RadioButton;
    if (radio != null)
    {
        GenderTextBlock.Text = $"Выбрано: {radio.Content}";
    }
}
```



Оба переключателя имеют одинаковое **GroupName="Gender"**, поэтому они взаимоисключают друг друга. Если не указывать **GroupName**, группа формируется внутри общего родителя (здесь **StackPanel**).

Событие **Checked** у обоих привязано к одному обработчику, где через параметр **sender** получаем конкретный **RadioButton** и его содержимое

При запуске ни один не выбран; можно задать начальное состояние, установив **IsChecked="True"** для одного из них.

Если переключатели находятся в разных контейнерах, но должны быть в одной группе, обязательно используйте **GroupName**.

Для сброса выбора можно установить **IsChecked = false** для всех, но обычно это не нужно, так как всегда должен быть выбран хотя бы один вариант.

Элемент **ComboBox**

ComboBox — это комбинированный элемент, который объединяет текстовое поле (или неизменяемый текст) и выпадающий список. Пользователь может выбрать элемент из списка или, в зависимости от режима, ввести своё значение. Экономит место, так как список скрыт до щелчка.

Основные свойства и события

Команда	Тип	Описание
ItemsSource	свойство	Привязывает коллекцию элементов для отображения.
SelectedItem	свойство	Получает или задаёт выбранный объект из списка.
SelectedValue	свойство	Получает или задаёт значение выбранного элемента, используя SelectedValuePath .
SelectedIndex	свойство	Индекс выбранного элемента (начиная с 0; -1, если ничего не выбрано).

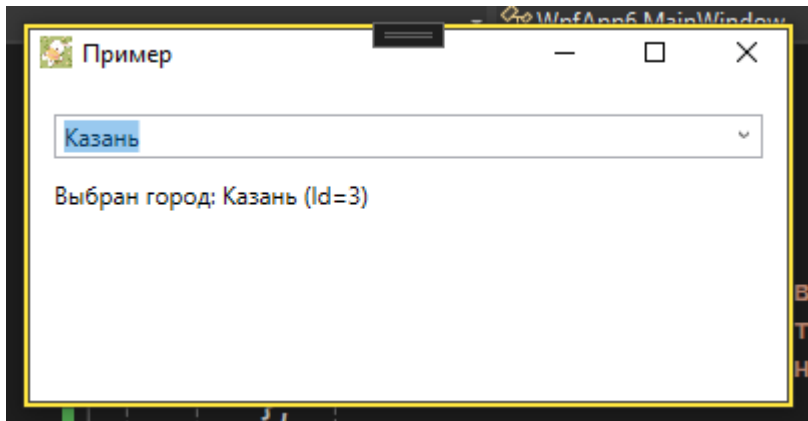
DisplayMemberPath	свойство	Указывает имя свойства, которое будет отображаться в списке.
IsEditable	свойство	Разрешает пользователю вводить текст в поле (если true).
Text	свойство	Текст, отображаемый в поле (при IsEditable=true можно получить введенный текст).
IsDropDownOpen	свойство	Управляет видимостью выпадающего списка.
SelectionChanged	событие	Возникает при изменении выбранного элемента.

Задача: Создать выпадающий список для выбора города. Сделать возможность ввода нового города (если его нет в списке) через IsEditable.

XAML-разметка:

```
<StackPanel Margin="10">
    <ComboBox x:Name="CityComboBox"
        IsEditable="True"
        DisplayMemberPath="Name"
        SelectedValuePath="Id"
        SelectionChanged="CityComboBox_SelectionChanged"
        Margin="5"/>
    <TextBlock x:Name="CityTextBlock" Margin="5"/>
</StackPanel>
</Window>
```

```
21 public partial class MainWindow : Window
22 {
23
24     public class City
25     {
26         public int Id { get; set; }
27         public string Name { get; set; }
28     }
29
30     public MainWindow()
31     {
32         InitializeComponent();
33         var cities = new List<City>
34         {
35             new City { Id = 1, Name = "Москва" },
36             new City { Id = 2, Name = "Санкт-Петербург" },
37             new City { Id = 3, Name = "Казань" }
38         };
39         CityComboBox.ItemsSource = cities;
40     }
41
42     private void CityComboBox_SelectionChanged(object sender, SelectionChangedEventArgs e)
43     {
44         if (CityComboBox.SelectedItem is City selectedCity)
45         {
46             CityTextBlock.Text = $"Выбран город: {selectedCity.Name} (Id={selectedCity.Id})";
47         }
48         else if (CityComboBox.SelectedItem == null && CityComboBox.IsEditable)
49         {
50             // Если IsEditable=true, SelectedItem может быть null, а Text содержит введенное значение
51             string enteredText = CityComboBox.Text;
52             if (!string.IsNullOrWhiteSpace(enteredText))
53             {
54                 CityTextBlock.Text = $"Введен новый город: {enteredText}";
55             }
56             else
57             {
58                 CityTextBlock.Text = "";
59             }
60         }
61     }
62 }
63 }
```



ItemsSource привязан к списку объектов City. DisplayMemberPath указывает, что показывать нужно свойство Name.

IsEditable="True" позволяет пользователю вводить текст. При вводе текста, которого нет в списке, SelectedItem становится null, а свойство Text содержит введённое значение.

В обработчике SelectionChanged мы проверяем: если есть SelectedItem (выбор из списка), отображаем его; если нет, но есть текст, значит, пользователь ввёл новое значение.

Для кастомизации отображения элементов используйте ItemTemplate.

При IsEditable="True" можно добавить автодополнение, но стандартно его нет; для этого нужно использовать сторонние библиотеки или реализовывать самому.

Свойство SelectedValuePath удобно, если нужно получать только идентификатор выбранного элемента.

Элемент ListBox

ListBox отображает список элементов и позволяет пользователю выбрать один или несколько (в зависимости от режима выбора). Элементы могут быть простыми строками или сложными объектами с шаблонами. ListBox поддерживает виртуализацию для больших коллекций.

Основные свойства и события

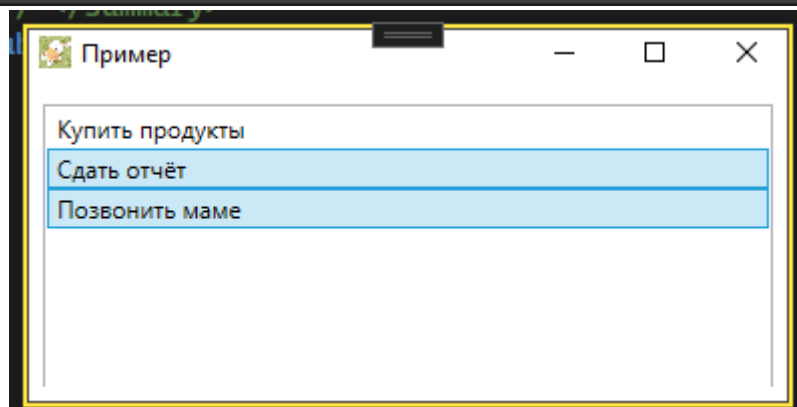
Команда	Тип	Описание
ItemsSource	свойство	Привязывает коллекцию данных.
SelectedItem	свойство	Получает или задаёт выбранный элемент.
SelectedItems	свойство	Коллекция выбранных элементов (при SelectionMode отличным от Single).
SelectedIndex	свойство	Индекс выбранного элемента (первого, если множественный выбор).
SelectionMode	свойство	Режим выбора: Single, Multiple (простой множественный), Extended (множественный с Ctrl/Shift).
DisplayMemberPath	свойство	Имя свойства для отображения текста элемента.
ItemTemplate	свойство	Шаблон данных для настройки внешнего вида элементов.
SelectionChanged	событие	Возникает при изменении выбора.

Задача: Создать список задач с флажками (множественный выбор) и отображать количество выбранных задач.

XAML-разметка:

```
<StackPanel Margin="10">
    <ListBox x:Name="TasksListBox"
        SelectionMode="Multiple"
        DisplayMemberPath="Title"
        SelectionChanged="TasksListBox_SelectionChanged"
        Height="150"/>
    <TextBlock x:Name="SelectedCountTextBlock" Margin="0,10,0,0"/>
</StackPanel>
</Window>
```

```
17 namespace WpfApp6
18 {
19     /// <summary>
20     /// Логика взаимодействия для MainWindow.xaml
21     /// </summary>
22     public partial class MainWindow : Window
23     {
24
25         public class TaskItem
26         {
27             public string Title { get; set; }
28             public bool IsCompleted { get; set; }
29         }
30
31         public MainWindow()
32         {
33             InitializeComponent();
34
35             var tasks = new ObservableCollection<TaskItem>
36             {
37                 new TaskItem { Title = "Купить продукты", IsCompleted = false },
38                 new TaskItem { Title = "Сдать отчёт", IsCompleted = false },
39                 new TaskItem { Title = "Позвонить маме", IsCompleted = true }
40             };
41             TasksListBox.ItemsSource = tasks;
42         }
43
44         private void TasksListBox_SelectionChanged(object sender, SelectionChangedEventArgs e)
45         {
46             SelectedCountTextBlock.Text = $"Выбрано задач: {TasksListBox.SelectedItems.Count}";
47         }
48     }
49 }
```



SelectionMode="Multiple" позволяет выбирать несколько элементов щелчками мыши (без модификаторов).

DisplayMemberPath="Title" отображает текст задачи.

В обработчике `SelectionChanged` мы просто считаем количество выбранных элементов.

Для множественного выбора используйте `SelectionMode="Extended"`, чтобы работали стандартные клавиши `Ctrl` и `Shift`.

`ListBox` поддерживает виртуализацию по умолчанию, если поместить его в контейнер с ограниченной высотой.